

ĐẠI HỌC BÁCH KHOA HÀ NỘI

ĐỒ ÁN TỐT NGHIỆP

**Hiển thị giao thông trong không gian 3D từ dữ liệu
mô phỏng SUMO**

ĐẶNG MINH HOÀNG

Hoang.DM225719@sis.hust.edu.vn

Chương trình đào tạo: Công nghệ thông tin Việt-Nhật

Giảng viên hướng dẫn: ThS. Nguyễn Tiến Thành

Khoa: Kỹ thuật máy tính

Trường: Công nghệ thông tin và Truyền thông

HÀ NỘI, 02/2025

ĐẠI HỌC BÁCH KHOA HÀ NỘI

ĐỒ ÁN TỐT NGHIỆP

Hiển thị giao thông trong không gian 3D từ dữ liệu
mô phỏng SUMO

ĐẶNG MINH HOÀNG

Hoang.DM225719@sis.hust.edu.vn

Chương trình đào tạo: Công nghệ thông tin Việt-Nhật

Giảng viên hướng dẫn: ThS. Nguyễn Tiến Thành _____

Chữ kí GVHD

Khoa: Kỹ thuật máy tính

Trường: Công nghệ Thông tin và Truyền thông

HÀ NỘI, 02/2025

LỜI CẢM ƠN

Trong quá trình thực hiện đồ án tốt nghiệp, em đã nhận được nhiều sự giúp đỡ và góp ý quý báu. Trước hết, em xin chân thành cảm ơn ThS. Nguyễn Tiến Thành đã tận tình hướng dẫn, định hướng nội dung và hỗ trợ em tháo gỡ các vướng mắc trong suốt quá trình nghiên cứu và triển khai. Em cũng xin cảm ơn các thầy cô của Trường Công nghệ Thông tin và Truyền thông — Đại học Bách khoa Hà Nội vì những kiến thức nền tảng và phương pháp học tập đã trang bị cho em. Em xin gửi lời cảm ơn tới gia đình và bạn bè vì luôn động viên, tạo điều kiện về thời gian và tinh thần để em hoàn thành đồ án đúng tiến độ. Những thiếu sót trong báo cáo là trách nhiệm của riêng em và em mong nhận được thêm góp ý để hoàn thiện hơn.

TÓM TẮT NỘI DUNG ĐỒ ÁN

Nhu cầu quan sát và phân tích mô phỏng giao thông ngày càng tăng, tuy nhiên giao diện 2D truyền thống của SUMO còn hạn chế về khả năng trực quan hóa không gian và theo dõi đồng thời nhiều đối tượng. Một số giải pháp hiển thị 3D đã tồn tại, nhưng thường phụ thuộc vào bản đồ tĩnh, khó tái sử dụng cho nhiều kịch bản và bị giới hạn về số lượng, loại đối tượng giao thông có thể thể hiện. Trong đồ án này, em lựa chọn hướng tiếp cận chuyển đổi dữ liệu mô phỏng từ SUMO sang môi trường hiển thị 3D bằng cách đọc các tệp XML (mạng đường, tuyến, phương tiện, tín hiệu) và dựng cảnh trong Unity3D. Cách tiếp cận này được chọn vì tận dụng được năng lực mô phỏng mạnh của SUMO, đồng thời khai thác pipeline dựng hình và tương tác của Unity3D để tạo một hệ thống hiển thị linh hoạt, có thể mở rộng theo dữ liệu đầu vào. Trên cơ sở đó, đồ án xây dựng quy trình gồm phân tích cấu trúc dữ liệu xuất từ SUMO, thiết kế mô-đun đọc và trích xuất thông tin hình học/topology của mạng, ánh xạ sang hệ tọa độ trong Unity, tạo các đối tượng 3D tương ứng (đường, phương tiện, đèn giao thông) và cập nhật trạng thái theo thời gian mô phỏng để kết xuất chuyển động. Đóng góp chính của đồ án là đề xuất và hiện thực hóa cơ chế dựng bản đồ và đối tượng giao thông dựa trên dữ liệu XML, giúp tự động hóa quá trình tạo cảnh 3D cho nhiều kịch bản; đồng thời xây dựng phiên bản thử nghiệm có thể hiển thị đường giao thông và phương tiện, hỗ trợ quan sát trực quan các tình huống mô phỏng. Kết quả thực nghiệm ban đầu cho thấy hệ thống tái hiện được chuyển động phương tiện trên bản đồ và đáp ứng nhu cầu quan sát, làm tiền đề cho các cải tiến tiếp theo về hiệu năng và chức năng phân tích.

Sinh viên thực hiện
(Ký và ghi rõ họ tên)

MỤC LỤC

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI.....	1
1.1 Đặt vấn đề.....	1
1.2 Mục tiêu và phạm vi đề tài.....	2
1.3 Định hướng giải pháp.....	3
1.4 Bố cục đồ án	4
CHƯƠNG 2. CÔNG NGHỆ SỬ DỤNG.....	5
2.1 Tổng quan công nghệ và vai trò trong hệ thống.....	5
2.2 SUMO (Simulation of Urban Mobility)	5
2.2.1 Giới thiệu	5
2.2.2 Lý do lựa chọn SUMO.....	6
2.2.3 Các thành phần cấu hình và dữ liệu trong SUMO.....	6
2.2.4 Các lựa chọn thay thế và đánh giá.....	7
2.3 TraCI (Traffic Control Interface).....	7
2.3.1 Giới thiệu	7
2.3.2 Mô hình hoạt động client-server	8
2.3.3 Nhóm chức năng TraCI sử dụng trong đồ án.....	8
2.3.4 Lựa chọn thay thế.....	8
2.4 Python cho điều phối và xử lý dữ liệu	8
2.4.1 Vai trò trong hệ thống.....	8
2.4.2 Chuẩn hoá theo hợp đồng dữ liệu độc lập nền tảng.....	9
2.4.3 Tổ chức xử lý theo bước thời gian	9
2.4.4 Lựa chọn thay thế.....	10
2.5 Unity cho hiển thị 3D và tương tác	10
2.5.1 Giới thiệu và lý do lựa chọn	10

2.5.2 Pipeline hiển thị đối tượng giao thông	10
2.5.3 Lựa chọn thay thế.....	11
2.6 Tích hợp SUMO–TraCI/Python–Unity.....	11
2.6.1 Phương thức trao đổi dữ liệu	11
2.6.2 Schema dữ liệu tối thiểu	11
2.6.3 Đồng bộ thời gian giữa timestep và frame.....	12
2.6.4 Tổng kết chương	12
CHƯƠNG 3. THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG	13
3.1 Thiết kế kiến trúc.....	13
3.1.1 Lựa chọn kiến trúc phần mềm	13
3.1.2 Thiết kế tổng quan.....	13
3.1.3 Thiết kế chi tiết gói	14
3.2 Thiết kế chi tiết.....	15
3.2.1 Thiết kế giao diện	15
3.2.2 Thiết kế lớp	15
3.2.3 Thiết kế cơ sở dữ liệu	16
3.3 Xây dựng ứng dụng.....	17
3.3.1 Thư viện và công cụ sử dụng.....	17
3.3.2 Kết quả đạt được	17
3.3.3 Minh họa các chức năng chính	18
3.4 Kiểm thử.....	18
3.5 Triển khai	19
CHƯƠNG 4. CÁC GIẢI PHÁP VÀ ĐÓNG GÓP NỔI BẬT.....	20
4.1 Giải pháp giao tiếp thời gian thực qua TCP: tách khung thông điệp bằng marker <END>	20
4.1.1 Bài toán/vấn đề	20

4.1.2 Giải pháp	20
4.1.3 Kết quả đạt được	21
4.2 Giải pháp điều phối vòng lặp mô phỏng, đa cổng kết nối và dừng hệ thống an toàn	21
4.2.1 Bài toán/vấn đề	21
4.2.2 Giải pháp	22
4.2.3 Kết quả đạt được	22
4.3 Giải pháp sinh mạng SUMO để benchmark theo phương pháp vét cạn từ lưới	22
4.3.1 Bài toán/vấn đề	22
4.3.2 Giải pháp	23
4.3.3 Kết quả đạt được	24
4.4 Giải pháp sinh tuyến đường (route) và lọc cặp điểm không liên thông.....	24
4.4.1 Bài toán/vấn đề	24
4.4.2 Giải pháp	24
4.4.3 Kết quả đạt được	25
4.5 Giải pháp chuẩn hóa dữ liệu TraCI cho hiển thị 3D (xe, đèn giao thông, người đi bộ)	25
4.5.1 Bài toán/vấn đề	25
4.5.2 Giải pháp	25
4.5.3 Kết quả đạt được	26
4.6 Giải pháp ghi log và tổng hợp kết quả tự động theo lần chạy	26
4.6.1 Bài toán/vấn đề	26
4.6.2 Giải pháp	26
4.6.3 Kết quả đạt được	27
4.7 Kết chương.....	27

CHƯƠNG 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	28
5.1 Kết luận.....	28
5.2 Hướng phát triển.....	29
TÀI LIỆU THAM KHẢO.....	32
PHỤ LỤC.....	34
A. HƯỚNG DẪN VIẾT ĐỒ ÁN TỐT NGHIỆP	34
A.1 Ngành học.....	35
A.2 Đánh dấu (bullet) và đánh số (numering)	35
A.3 Cách thêm bảng	36
A.4 Chèn hình ảnh	36
A.5 Tài liệu tham khảo	37
A.6 Cách viết phương trình và công thức toán học.....	37
A.7 Qui cách đóng quyển.....	37
B. ĐẶC TẢ USE CASE.....	39
B.1 Đặc tả use case “Thống kê tình hình mượn sách”	39
B.2 Đặc tả use case “Đăng ký làm thẻ mượn”	39

DANH MỤC HÌNH VẼ

Hình 2.1	Kiến trúc tổng quan hệ thống mô phỏng và hiển thị 3D	5
Hình 2.2	Vòng lặp điều khiển và thu thập dữ liệu bằng TraCI	8
Hình 3.1	Kiến trúc tổng quan của hệ thống mô phỏng–hiển thị	14
Hình 3.2	Minh họa trực quan mô phỏng trên Unity	18
Hình 3.3	Minh họa mô phỏng trên SUMO-GUI	18
Hình A.1	Internet vạn vật	36
Hình A.2	Quy cách đóng quyển đồ án	38
Hình A.3	Quy cách đóng quyển đồ án	38

DANH MỤC BẢNG BIỂU

Bảng 2.1	Đầu vào/đầu ra của SUMO trong hệ thống (tóm tắt)	7
Bảng 2.2	Schema dữ liệu tối thiểu trao đổi giữa Python và Unity	11
Bảng 3.1	Danh sách thư viện và công cụ sử dụng	17
Bảng 3.2	Một số trường hợp kiểm thử chính	19
Bảng A.1	Table to test captions and labels.	36

DANH MỤC THUẬT NGỮ VÀ TỪ VIẾT TẮT

Thuật ngữ	Ý nghĩa
API	Giao diện lập trình ứng dụng (Application Programming Interface)
EUD	Phát triển ứng dụng người dùng cuối(End-User Development)
SUMO	Trình mô phỏng giao thông đô thị mã nguồn mở (Simulation of Urban Mobility)
TraCI	Giao diện điều khiển mô phỏng giao thông (Traffic Control Interface)

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI

1.1 Đặt vấn đề

Trong bối cảnh đô thị hóa nhanh và mật độ phương tiện gia tăng, bài toán tổ chức và quản lý giao thông ngày càng trở nên phức tạp, đòi hỏi các phương pháp phân tích có căn cứ dữ liệu để hỗ trợ quy hoạch, đánh giá phương án tổ chức giao thông và kiểm chứng các giả thuyết kỹ thuật. Trên thực tế, việc thử nghiệm trực tiếp ngoài hiện trường thường tốn kém, khó kiểm soát biến số, và có thể tiềm ẩn rủi ro về an toàn. Vì vậy, mô phỏng giao thông trở thành một công cụ quan trọng, giúp tái hiện nhiều kịch bản khác nhau trong điều kiện có thể kiểm soát, từ đó hỗ trợ quá trình ra quyết định.

Trong nhóm các công cụ mô phỏng, SUMO (Simulation of Urban MObility) là một nền tảng mô phỏng vi mô được sử dụng rộng rãi, cho phép mô tả mạng đường, luồng phương tiện và cơ chế điều khiển (như tín hiệu đèn) với mức độ chi tiết cao. Kết quả mô phỏng có thể cung cấp dữ liệu theo thời gian về vị trí, tốc độ, hướng di chuyển của phương tiện, cũng như các thông tin liên quan đến trạng thái mạng. Tuy nhiên, dữ liệu đầu ra này thường có khối lượng lớn và giàu tính kỹ thuật, khiến việc quan sát và khai thác hiệu quả trở thành một thách thức, đặc biệt khi người dùng cần đánh giá trực quan cấu trúc không gian hoặc diễn biến ở các khu vực có hình học phức tạp.

Việc quan sát kết quả mô phỏng bằng giao diện 2D truyền thống thường đáp ứng tốt các thao tác cơ bản, nhưng vẫn gặp hạn chế khi cần: (i) nhận thức không gian tại các nút giao nhiều nhánh hoặc có tổ chức làn phức tạp; (ii) theo dõi đồng thời nhiều đối tượng trong bối cảnh mật độ cao; và (iii) trình bày kết quả cho nhóm người dùng không chuyên hoặc trong các hoạt động trao đổi, thuyết minh phương án. Trong các trường hợp này, góc nhìn 3D có thể giúp người quan sát nắm bắt tốt hơn mối quan hệ không gian giữa đường, làn, phương tiện và tín hiệu điều khiển, đồng thời tạo điều kiện cho các hình thức tương tác và trình bày trực quan.

Mặc dù vậy, các giải pháp hiển thị 3D trong thực tế thường gặp những rào cản như phụ thuộc vào bản đồ dựng sẵn (tĩnh), cần nhiều thao tác thủ công khi thay đổi kịch bản, hoặc khó mở rộng khi số lượng đối tượng và loại đối tượng tăng lên. Ngoài ra, việc chuyển dữ liệu mô phỏng sang môi trường 3D phát sinh các vấn đề kỹ thuật như ánh xạ hình học từ mô tả mạng đường (đa tuyến) sang mô hình cảnh, chuyển đổi hệ tọa độ, đồng bộ thời gian giữa bước mô phỏng và khung hình hiển thị, và đảm bảo hiệu năng khi cập nhật số lượng lớn đối tượng theo thời gian.

Do đó, việc xây dựng một hệ thống hiển thị giao thông 3D dựa trên dữ liệu đầu ra của SUMO, có khả năng tự động dựng cảnh theo bản đồ mô phỏng và cập nhật trạng thái theo thời gian, là cần thiết để nâng cao hiệu quả quan sát, phân tích và truyền đạt kết quả mô phỏng. Hệ thống này đóng vai trò cầu nối giữa dữ liệu mô phỏng (mang tính kỹ thuật) và nhu cầu khai thác trực quan, giúp người dùng tiếp cận, kiểm chứng và trình bày kịch bản giao thông một cách trực quan và nhất quán.

1.2 Mục tiêu và phạm vi đề tài

Từ các vấn đề nêu ở phần 1.1, đề án hướng tới mục tiêu xây dựng một hệ thống hiển thị mô phỏng giao thông trong không gian 3D, trong đó dữ liệu đầu vào được lấy trực tiếp từ các tệp xuất ra của SUMO và/hoặc từ quá trình chạy mô phỏng. Trọng tâm của hệ thống là hình thành một quy trình chuyển đổi dữ liệu mạng đường và dữ liệu mô phỏng sang các đối tượng 3D trong Unity3D một cách nhất quán, từ đó hỗ trợ quan sát diễn biến giao thông theo thời gian trong một môi trường trực quan hơn so với 2D.

Về mục tiêu cụ thể, đề án tập trung vào ba nhóm mục tiêu chính. Thứ nhất, xây dựng cơ chế nhập và phân tích dữ liệu mô tả mạng đường của SUMO (tệp `.net.xml`) để phục hồi hình học và cấu trúc topo cơ bản của mạng trong môi trường 3D, bảo đảm các đối tượng được dựng bám sát dữ liệu nguồn. Thứ hai, xây dựng cơ chế thu thập dữ liệu động theo thời gian của mô phỏng (thông qua TraCI hoặc tệp dữ liệu theo thời gian nếu có), nhằm lấy được trạng thái phương tiện và một số thành phần điều khiển (ví dụ tín hiệu đèn) theo từng bước mô phỏng. Thứ ba, thiết kế cơ chế cập nhật và hiển thị trạng thái trong Unity3D theo thời gian, bảo đảm chuyển động thể hiện đúng quỹ đạo tổng quát, đồng thời duy trì khả năng hiển thị ổn định khi số lượng đối tượng tăng.

Trong phạm vi của đề tài, hệ thống tập trung vào các thành phần cốt lõi gồm: (i) dựng cảnh 3D từ dữ liệu mạng đường, bao gồm các đoạn đường và các yếu tố hình học phục vụ quan sát; (ii) biểu diễn phương tiện như các thực thể di chuyển theo trạng thái thời gian; (iii) biểu diễn các tín hiệu điều khiển ở mức phù hợp với dữ liệu có thể truy xuất; và (iv) chuẩn hóa, ánh xạ dữ liệu sang hệ tọa độ của Unity3D để bảo đảm tính nhất quán trong hiển thị. Phần đánh giá được thực hiện thông qua các kịch bản mô phỏng mẫu, nhằm kiểm chứng tính đúng đắn của việc ánh xạ dữ liệu, khả năng tái hiện diễn biến theo thời gian và tính ổn định của hiển thị.

Ngoài phạm vi của đề án là các bài toán tối ưu điều khiển giao thông, hiệu chỉnh/tham số hóa mô hình mô phỏng của SUMO, hoặc phát triển thuật toán ra quyết định điều khiển phương tiện. Trong phạm vi này, hệ thống đóng vai trò là lớp trực quan hóa và hỗ trợ phân tích dựa trên dữ liệu mô phỏng sẵn có, qua đó góp

phần tăng khả năng quan sát và trình bày kịch bản thay vì can thiệp vào thuật toán mô phỏng.

1.3 Định hướng giải pháp

Để hiện thực hóa mục tiêu ở phần 1.2, đề án lựa chọn hướng tiếp cận dựa trên chuỗi công cụ: SUMO dùng để mô phỏng giao thông vi mô và sinh dữ liệu; Python kết hợp TraCI dùng để truy xuất trạng thái mô phỏng theo thời gian; Unity3D dùng để dựng cảnh và hiển thị 3D. Lựa chọn này phù hợp vì SUMO cung cấp dữ liệu mô phỏng có cấu trúc rõ ràng và phổ biến (XML/TraCI), trong khi Unity3D là môi trường phát triển 3D mạnh, hỗ trợ dựng cảnh, quản lý đối tượng, camera và tương tác, qua đó nâng cao khả năng quan sát trực quan.

Trên cơ sở đó, giải pháp được triển khai theo một quy trình tổng quát gồm các thành phần chính. Thứ nhất, mô-đun phân tích mạng đọc tệp `.net.xml` để trích xuất hình học của các đoạn đường và làn (thường được biểu diễn dưới dạng đa tuyến), cũng như một số quan hệ cấu trúc cần thiết phục vụ dựng cảnh. Kết quả được chuyển thành một biểu diễn trung gian phù hợp để sinh các đối tượng trong Unity3D. Thứ hai, mô-đun thu thập dữ liệu động kết nối với phiên mô phỏng SUMO thông qua TraCI để lấy trạng thái phương tiện theo từng bước mô phỏng, bao gồm vị trí, hướng chuyển động và các thuộc tính trạng thái cơ bản. Thứ ba, mô-đun ánh xạ và đồng bộ chuyển đổi hệ tọa độ từ không gian mô phỏng sang không gian Unity3D, đồng thời tổ chức cơ chế đồng bộ thời gian giữa bước mô phỏng và chu kỳ cập nhật khung hình nhằm giảm sai lệch cảm nhận khi hiển thị.

Trong quá trình triển khai, một yêu cầu quan trọng là bảo đảm tính nhất quán của dữ liệu giữa các mô-đun: dữ liệu hình học mạng đường cần dùng chung chuẩn tọa độ với dữ liệu phương tiện; các đối tượng được tạo ra trong Unity3D cần có vòng đời phù hợp với trạng thái mô phỏng (tạo mới khi phương tiện xuất hiện, cập nhật khi di chuyển và loại bỏ khi rời mạng). Đồng thời, để đảm bảo khả năng mở rộng, giải pháp ưu tiên tổ chức dữ liệu theo hướng tách biệt phần đọc/chuẩn hóa dữ liệu với phần hiển thị, giúp giảm phụ thuộc và thuận tiện khi thay đổi kịch bản mô phỏng.

Đóng góp chính của đề án là đề xuất và hiện thực một quy trình chuyển đổi dữ liệu mô phỏng từ SUMO sang môi trường hiển thị 3D trong Unity3D theo hướng tự động, giảm thao tác thủ công khi thay đổi kịch bản. Bên cạnh đó, đề án xây dựng được cơ chế dựng cảnh từ dữ liệu mạng đường và cơ chế cập nhật trạng thái phương tiện theo thời gian để tái hiện diễn biến giao thông trong không gian 3D. Kết quả thử nghiệm trên các kịch bản mô phỏng mẫu cho thấy hệ thống có thể dựng được mạng đường từ dữ liệu đầu vào và biểu diễn chuyển động phương tiện theo dữ liệu

thu từ SUMO một cách ổn định, tạo cơ sở cho việc quan sát và phân tích trực quan.

1.4 Bố cục đề án

Phần còn lại của báo cáo đề án tốt nghiệp này được tổ chức như sau: Chương 2 trình bày tổng quan các công cụ và nền tảng được sử dụng trong hệ thống, bao gồm SUMO, TraCI và Unity3D, đồng thời nêu các đặc trưng dữ liệu đầu vào/đầu ra liên quan đến quá trình mô phỏng và trực quan hóa. Chương 3 trình bày phân tích và thiết kế hệ thống, tập trung vào kiến trúc tổng thể, quy trình chuyển đổi dữ liệu và các mô-đun chính phục vụ dựng cảnh và cập nhật trạng thái theo thời gian. Chương 4 trình bày kết quả thực nghiệm, mô tả các kịch bản mô phỏng mẫu, kết quả dựng cảnh và quan sát, đồng thời thảo luận một số đánh giá về tính đúng đắn và tính ổn định của hiển thị. Cuối cùng, Chương 5 tổng kết các kết quả đạt được, các đóng góp chính của đề án và đề xuất hướng phát triển trong tương lai.

CHƯƠNG 2. CÔNG NGHỆ SỬ DỤNG

2.1 Tổng quan công nghệ và vai trò trong hệ thống

Trong đồ án, bài toán cần giải quyết là hiển thị giao thông trong không gian 3D dựa trên dữ liệu mô phỏng. Để đạt được mục tiêu này, hệ thống cần đồng thời đáp ứng ba nhóm yêu cầu chính: (i) mô phỏng giao thông có tính thực tế theo thời gian, (ii) trích xuất và (khi cần) điều khiển mô phỏng theo từng bước thời gian, và (iii) trực quan hoá kết quả dưới dạng 3D, có khả năng quan sát và tương tác.

Ngăn xếp công nghệ được lựa chọn bao gồm bốn thành phần chính:

- **SUMO** (Simulation of Urban Mobility): nền tảng mô phỏng giao thông đô thị mã nguồn mở, hỗ trợ mô phỏng vi mô theo từng phương tiện.
- **TraCI** (Traffic Control Interface): giao diện/protocol cho phép chương trình bên ngoài kết nối và điều khiển SUMO theo *timestep*.
- **Python**: lớp điều phối (orchestration) để khởi tạo mô phỏng, giao tiếp qua TraCI, và xuất dữ liệu theo một *chuẩn trao đổi* độc lập nền tảng hiển thị.
- **Unity**: môi trường dựng cảnh và hiển thị 3D theo thời gian thực, đảm nhận phần trực quan hoá phương tiện và hạ tầng.

Về mặt kiến trúc, luồng dữ liệu tổng quát có thể mô tả như sau: SUMO mô phỏng giao thông và cung cấp trạng thái đối tượng theo thời gian; Python sử dụng TraCI để lấy trạng thái này theo từng bước thời gian và xuất dữ liệu theo một định dạng chuẩn (hợp đồng dữ liệu) không phụ thuộc Unity; Unity (hoặc một tầng hiển thị khác trong tương lai) đọc dữ liệu chuẩn và thực hiện các bước chuyển đổi phục vụ việc render (ví dụ ánh xạ hệ toạ độ, nội suy).

TODO: Chèn hình kiến trúc tổng quan: **SUMO** ↔ **TraCI/Python** → **Unity**.
Gợi ý: vẽ dạng khối, ghi rõ dữ liệu trao đổi (ID xe, vị trí, tốc độ, hướng, trạng thái đèn).

Hình 2.1: Kiến trúc tổng quan hệ thống mô phỏng và hiển thị 3D

2.2 SUMO (Simulation of Urban Mobility)

2.2.1 Giới thiệu

SUMO là công cụ mô phỏng giao thông đô thị mã nguồn mở, được thiết kế để mô phỏng giao thông theo mô hình vi mô (microscopic simulation). Trong mô hình

vi mô, mỗi phương tiện (xe máy, ô tô, xe buýt, ...) được xem như một thực thể có trạng thái riêng (vị trí, vận tốc, gia tốc, hướng di chuyển) và tuân theo các quy tắc chuyển động/tuân thủ làn đường/điều khiển giao lộ. Cách tiếp cận này đặc biệt phù hợp khi mục tiêu là tái hiện chuyển động phương tiện một cách liên tục theo thời gian và phục vụ trực quan hoá.

Trong phạm vi đồ án, SUMO được sử dụng để tạo ra dữ liệu mô phỏng nhất quán theo thời gian. Thay vì chỉ xuất báo cáo tổng kê tổng hợp, hệ thống cần lấy trạng thái chi tiết của từng phương tiện theo *timestep* để có thể hiển thị chuyển động trong Unity.

2.2.2 Lý do lựa chọn SUMO

SUMO được lựa chọn vì các lý do sau:

- **Phù hợp bài toán mô phỏng đô thị:** hỗ trợ mạng đường dạng đồ thị (nút–cạnh), mô phỏng làn đường, giao lộ, tín hiệu đèn.
- **Khả năng điều khiển thời gian thực:** kết hợp với TraCI cho phép chạy mô phỏng theo từng bước thời gian và truy vấn trạng thái tức thời.
- **Dễ tích hợp:** định dạng XML rõ ràng, hệ sinh thái công cụ hỗ trợ tạo mạng/route; phù hợp để Python xử lý và tự động hoá.
- **Mã nguồn mở:** thuận tiện cho mục tiêu học thuật và triển khai trong môi trường đồ án.

2.2.3 Các thành phần cấu hình và dữ liệu trong SUMO

Để chạy một kịch bản mô phỏng, SUMO thường cần ba nhóm tệp chính: tệp mô tả **mạng đường**, tệp mô tả **luồng/tuyến phương tiện**, và tệp **cấu hình mô phỏng**. Trong đồ án, các nhóm tệp này cũng là đầu vào/đầu ra quan trọng để đảm bảo tính lặp lại của thí nghiệm.

Mạng đường (network). Mạng đường thể hiện hạ tầng giao thông ở dạng đồ thị có hướng: nút (junction/node) và cạnh (edge). Tùy cách xây dựng mạng, có thể bắt đầu từ các tệp rời như `.nod.xml` và `.edg.xml`, sau đó biên dịch thành `.net.xml`. Tệp `.net.xml` là biểu diễn mạng ở mức chi tiết đủ để mô phỏng (bao gồm làn đường, toạ độ, kết nối ở nút giao, ...).

Luồng/tuyến (routes). Luồng phương tiện có thể được mô tả bằng tệp `.rou.xml`, trong đó định nghĩa tuyến đường (route), loại phương tiện (vehicle type) và lịch khởi hành (depart time). Khi mục tiêu là tái hiện giao thông có mật độ thay đổi theo thời gian, cách xây dựng route cần có khả năng điều chỉnh tham số như lưu lượng đầu vào, phân bố tuyến, hoặc kịch bản đặc thù (giờ cao điểm).

Cấu hình chạy (configuration). Tập `.sumocfg` tập hợp các thành phần trên và chỉ định tham số mô phỏng: thời gian bắt đầu/kết thúc, độ dài bước thời gian (*step-length*), chế độ xuất dữ liệu, Việc chuẩn hoá tham số trong tập cấu hình giúp các thí nghiệm có thể chạy lại và so sánh.

Bảng 2.1: Đầu vào/đầu ra của SUMO trong hệ thống (tóm tắt)

Nhóm	Mô tả
Đầu vào	Mạng đường (<code>.net.xml</code>), luồng/tuyến (<code>.rou.xml</code>), cấu hình (<code>.sumocfg</code>)
Đầu ra	Trạng thái theo thời gian: danh sách phương tiện, vị trí (x, y) , vận tốc, hướng; trạng thái đèn; thông tin edge/lane (nếu cần)

2.2.4 Các lựa chọn thay thế và đánh giá

Với bài toán mô phỏng giao thông, có thể xem xét nhiều công cụ khác nhau. Một số công cụ mô phỏng thương mại như Aimsun hoặc VISSIM có khả năng mô phỏng mạnh và nhiều module, tuy nhiên đi kèm hạn chế về giấy phép, chi phí và mức độ phù hợp trong bối cảnh triển khai đồ án. Ở chiều ngược lại, các công cụ mô phỏng vĩ mô (ví dụ MATSim) tập trung vào lưu lượng tổng thể và hành vi theo mô hình nhu cầu, không tối ưu cho việc lấy trạng thái chi tiết của từng phương tiện để hiển thị 3D liên tục. Ngoài ra, các môi trường 3D phục vụ xe tự hành (ví dụ CARLA) cung cấp mô phỏng hình ảnh và cảm biến, nhưng chi phí tích hợp và yêu cầu tài nguyên thường lớn hơn đáng kể so với mục tiêu hiển thị giao thông dựa trên dữ liệu mô phỏng.

2.3 TraCI (Traffic Control Interface)

2.3.1 Giới thiệu

TraCI là giao diện/protocol cho phép một chương trình bên ngoài (client) kết nối tới SUMO (server) để điều khiển mô phỏng theo từng bước thời gian. Thay vì để SUMO tự chạy độc lập và chỉ xuất log sau khi kết thúc, TraCI cho phép hệ thống:

- điều khiển tiến trình mô phỏng theo *timestep* (bước thời gian rời rạc),
- truy vấn trạng thái đối tượng ngay tại thời điểm đang chạy,
- thay đổi trạng thái/ra lệnh điều khiển (nếu kịch bản yêu cầu),
- đồng bộ dữ liệu mô phỏng với hệ thống hiển thị 3D.

Trong đồ án, TraCI là “cầu nối” quan trọng để Python có thể lấy dữ liệu theo thời gian thực và cung cấp cho Unity. Nhờ đó, Unity không cần đọc các tệp log hậu kỳ mà có thể cập nhật chuyển động theo nhịp mô phỏng.

2.3.2 Mô hình hoạt động client–server

Về nguyên lý, SUMO chạy như một server TraCI. Ứng dụng Python đóng vai trò client: khởi tạo kết nối, gửi lệnh `simulationStep` để tiến mô phỏng, sau đó truy vấn các đối tượng quan tâm (ví dụ danh sách xe đang tồn tại, vị trí của từng xe, trạng thái đèn tín hiệu). Mỗi vòng lặp tương ứng với một bước thời gian, từ đó hình thành luồng dữ liệu theo dạng chuỗi thời gian.

TODO: Chèn hình minh hoạ vòng lặp TraCI: connect → simulationStep → get state → export/send to Unity.

Hình 2.2: Vòng lặp điều khiển và thu thập dữ liệu bằng TraCI

2.3.3 Nhóm chức năng TraCI sử dụng trong đồ án

Tuỳ theo kịch bản và dữ liệu cần hiển thị, TraCI cung cấp nhiều nhóm hàm truy vấn/điều khiển. Trong đồ án hiển thị 3D, các nhóm chức năng thường dùng gồm:

- **simulation**: điều khiển tiến trình mô phỏng, lấy thời gian mô phỏng, số xe đang tồn tại.
- **vehicle**: lấy và cập nhật trạng thái xe (toạ độ, tốc độ, hướng), truy vấn tuyến hiện tại, làn hiện tại.
- **trafficlight** (nếu có): lấy/đặt pha đèn, thời gian còn lại của pha.
- **edge/lane**: lấy thông tin hạ tầng (độ dài, tốc độ giới hạn) và các thống kê cục bộ (nếu phục vụ hiển thị/đánh giá).

2.3.4 Lựa chọn thay thế

Một cách tiếp cận thay thế là chạy SUMO và xuất log sau mô phỏng (ví dụ dạng CSV/FCD), rồi để Unity đọc lại và phát lại như một đoạn phim. Cách này có ưu điểm đơn giản, dễ debug, nhưng hạn chế khả năng đồng bộ theo thời gian thực và khó mở rộng sang các tình huống cần can thiệp mô phỏng trong lúc chạy (ví dụ thay đổi chính sách đèn, điều tiết xe theo sự kiện). Do đó, TraCI phù hợp hơn với mục tiêu “kết nối mô phỏng–hiển thị” trong đồ án.

2.4 Python cho điều phối và xử lý dữ liệu

2.4.1 Vai trò trong hệ thống

Python đóng vai trò lớp điều phối giữa mô phỏng (SUMO) và hiển thị (Unity). Ở mức hệ thống, Python chịu trách nhiệm:

- khởi tạo phiên làm việc với SUMO (chạy SUMO/GUI, cấu hình cổng TraCI),
- điều khiển mô phỏng theo từng bước thời gian thông qua TraCI,
- thu thập dữ liệu cần thiết (phương tiện, trạng thái đèn, ...),
- chuẩn hoá dữ liệu theo **chuẩn trao đổi độc lập nền tảng** (lọc trường không cần thiết, thống nhất đơn vị/định dạng và cấu trúc bản ghi),
- xuất/gửi dữ liệu theo cơ chế trao đổi đã chọn.

So với việc viết toàn bộ lớp điều phối trong Unity/C#, Python mang lại lợi thế về tốc độ phát triển, khả năng thử nghiệm nhanh, và thuận tiện cho việc viết script tự động tạo dữ liệu (tạo map, tạo route, chạy hàng loạt kịch bản).

2.4.2 Chuẩn hoá theo hợp đồng dữ liệu độc lập nền tảng

Một mục tiêu quan trọng của hệ thống là tách biệt tầng mô phỏng/thu thập dữ liệu (Python) khỏi tầng hiển thị (Unity). Do đó, Python **không thực hiện các chuyển đổi đặc thù Unity** như ánh xạ toạ độ 2D của SUMO sang trục 3D của Unity hay chuyển đổi rotation theo quy ước engine. Thay vào đó, Python xuất dữ liệu theo một **hợp đồng dữ liệu chuẩn** (data contract) có thể tái sử dụng bởi bất kỳ nền tảng đồ hoạ nào.

Hợp đồng dữ liệu chuẩn cần thống nhất các nội dung tối thiểu sau:

- **Đơn vị đo:** ví dụ vị trí theo mét, thời gian theo giây, vận tốc theo m/s.
- **Hệ toạ độ nguồn:** giữ nguyên hệ toạ độ của SUMO (2D) và cung cấp mô tả/metadata cần thiết để tầng hiển thị tự ánh xạ.
- **Trường dữ liệu bắt buộc:** `vehicleId`, `t`, `x`, `y`, `speed`, `heading` (và các trường tùy chọn nếu cần).
- **Định dạng bản ghi:** cấu trúc dòng/bản ghi (CSV/JSON) và quy ước về dấu phân cách, encoding, thứ tự trường.

Nhờ cách thiết kế này, khi thay đổi nền tảng hiển thị (ví dụ chuyển từ Unity sang một engine khác), phần Python có thể giữ nguyên; chỉ cần xây dựng một tầng “adapter hiển thị” mới để đọc dữ liệu chuẩn và thực hiện các chuyển đổi phù hợp với engine.

2.4.3 Tổ chức xử lý theo bước thời gian

Một chiến lược phổ biến là Python chạy vòng lặp theo timestep: mỗi bước tiến mô phỏng một khoảng Δt , sau đó lấy trạng thái và gửi sang Unity. Trong các bước này, có thể áp dụng tối ưu như chỉ gửi các phương tiện đang tồn tại, hoặc chỉ gửi những thay đổi so với bước trước. Ngoài ra, để chuyển động mượt trong Unity, có

thể lựa chọn gửi dữ liệu ở tần suất thấp hơn và nội suy trong Unity.

2.4.4 Lựa chọn thay thế

Về mặt kỹ thuật, TraCI cũng hỗ trợ các client trong Java/C++ và có thể xây dựng lớp điều phối bằng các ngôn ngữ này. Tuy nhiên, với đồ án ứng dụng, Python là lựa chọn cân bằng giữa tính linh hoạt, khả năng xử lý dữ liệu và thời gian phát triển. Một lựa chọn khác là viết trực tiếp lớp giao tiếp trong Unity/C#, giúp giảm một tầng trung gian, nhưng làm tăng độ phức tạp triển khai và khó tái sử dụng các script xử lý dữ liệu/phân tích.

2.5 Unity cho hiển thị 3D và tương tác

2.5.1 Giới thiệu và lý do lựa chọn

Unity là nền tảng phát triển ứng dụng 3D thời gian thực, hỗ trợ dựng cảnh, điều khiển camera, chiếu sáng và xây dựng giao diện tương tác. Trong đồ án, Unity đảm nhiệm vai trò trực quan hoá giao thông 3D dựa trên dữ liệu mô phỏng từ SUMO. Việc hiển thị 3D cho phép người dùng quan sát trực quan mật độ xe, dòng xe qua nút giao, và hành vi di chuyển theo thời gian, từ đó hỗ trợ phân tích và trình bày kết quả.

Unity được lựa chọn vì các lý do chính: (i) hệ sinh thái phong phú cho dựng cảnh và xử lý chuyển động, (ii) khả năng cập nhật đối tượng theo thời gian thực, và (iii) phù hợp với yêu cầu triển khai nhanh trong đồ án.

2.5.2 Pipeline hiển thị đối tượng giao thông

Để hiển thị một mô phỏng giao thông trong Unity, hệ thống cần quản lý hai nhóm đối tượng: **hạ tầng** (đường, làn, nút giao) và **đối tượng động** (phương tiện, có thể mở rộng sang người đi bộ nếu kịch bản có). Một pipeline hiển thị thường gồm các bước:

- **Khởi tạo hạ tầng:** dựng mặt đường, làn đường và các điểm mốc dựa trên dữ liệu mạng (từ SUMO hoặc dữ liệu chuyển đổi trung gian).
- **Khởi tạo phương tiện:** tạo đối tượng xe theo danh sách ID, gán mô hình (mesh/prefab) và thuộc tính hiển thị.
- **Cập nhật theo thời gian:** tại mỗi bước, cập nhật vị trí và hướng xe dựa trên dữ liệu nhận từ Python.

Trong trường hợp số lượng xe lớn, một vấn đề quan trọng là hiệu năng. Một số kỹ thuật thường dùng gồm tái sử dụng đối tượng (object pooling), giảm số lần tạo/hủy đối tượng, hoặc chỉ hiển thị trong phạm vi quan sát.

2.5.3 Lựa chọn thay thế

Các engine 3D khác như Unreal Engine có khả năng đồ hoạ mạnh, tuy nhiên thường yêu cầu tài nguyên lớn và chu trình phát triển có thể nặng hơn. Các giải pháp WebGL (Three.js) thuận lợi cho triển khai web nhưng cần tự xây nhiều thành phần engine/asset và tối ưu hiệu năng. Vì vậy Unity là lựa chọn phù hợp cho mục tiêu trực quan hoá 3D trong đồ án.

2.6 Tích hợp SUMO–TraCI/Python–Unity

2.6.1 Phương thức trao đổi dữ liệu

Trong dự án, dữ liệu được truyền theo thời gian thực bằng cơ chế **TCP/IP**. Cụ thể, Python đóng vai trò tiến trình phía gửi (server) và Unity đóng vai trò phía nhận, liên tục đọc gói dữ liệu và cập nhật trạng thái hiển thị.

Cách tiếp cận truyền trực tiếp qua TCP/IP có các ưu điểm chính: độ trễ thấp, đồng bộ tốt với tiến trình mô phỏng theo từng bước thời gian, và phù hợp với mục tiêu hiển thị “đồng hồ thời gian” theo mô phỏng. Ngoài ra, vì dữ liệu được đóng gói theo hợp đồng dữ liệu chuẩn, tầng truyền TCP/IP cũng có thể tái sử dụng nếu thay Unity bằng một nền tảng hiển thị khác trong tương lai.

Dữ liệu được gửi theo chu kỳ **mỗi timestep**; kích thước trung bình mỗi gói khoảng **128KB**; cơ chế đóng gói và giải mã sử dụng **JSON**.

2.6.2 Schema dữ liệu tối thiểu

Để hiển thị chuyển động phương tiện, Unity tối thiểu cần biết: định danh xe, vị trí, hướng và vận tốc tại mỗi mốc thời gian. Ngoài ra, để phục vụ phân tích và mở rộng, có thể kèm theo thông tin làn/edge hoặc trạng thái đèn.

Lưu ý rằng schema này được thiết kế theo hướng **độc lập engine**: các trường phản ánh trạng thái mô phỏng và hình học nguồn. Các bước chuyển đổi sang hệ trục 3D, tỉ lệ hiển thị, cũng như nội suy theo frame sẽ do tầng hiển thị (Unity) đảm nhiệm.

Bảng 2.2: Schema dữ liệu tối thiểu trao đổi giữa Python và Unity

Trường dữ liệu	Ý nghĩa
vehicleId	Định danh phương tiện (duy nhất trong mô phỏng)
x, y	Toạ độ 2D trong hệ SUMO
speed	Vận tốc (m/s hoặc km/h theo quy ước)
(tuỳ chọn) laneId/edgeId	Làn/đoạn đường hiện tại để tô màu, thống kê, ...

2.6.3 Đồng bộ thời gian giữa timestep và frame

SUMO thường vận hành theo bước thời gian rời rạc (timestep) với độ dài Δt cố định. Trong khi đó Unity cập nhật theo số khung hình mỗi giây (FPS) và có thể thay đổi theo cấu hình máy. Điều này tạo ra bài toán đồng bộ: nếu Unity chỉ cập nhật vị trí xe đúng tại mỗi timestep, chuyển động có thể bị “giật” khi FPS cao. Vì vậy, có hai cách tiếp cận:

- **Cập nhật theo timestep:** đơn giản, đúng dữ liệu, phù hợp khi ưu tiên tính chính xác và dễ triển khai.
- **Nội suy theo frame:** Unity nhận hai trạng thái liên tiếp và nội suy vị trí/rotation cho các frame ở giữa, giúp chuyển động mượt hơn.

Trong dự án này, hệ thống lựa chọn **nội suy theo từng frame** ở tầng hiển thị (Unity) để đảm bảo chuyển động mượt và ổn định khi FPS cao.

Việc lựa chọn cách đồng bộ phụ thuộc vào mục tiêu của đồ án (chính xác tuyệt đối theo mô phỏng hay ưu tiên cảm nhận trực quan mượt mà) và quy mô số lượng xe hiển thị.

2.6.4 Tổng kết chương

Chương này đã trình bày các công nghệ chính được sử dụng trong đồ án gồm SUMO, TraCI, Python và Unity, cùng vai trò của từng thành phần trong kiến trúc tổng thể. Đồng thời, chương cũng phân tích cách tích hợp giữa mô phỏng và hiển thị 3D, đặc biệt là vấn đề trao đổi dữ liệu và đồng bộ thời gian, làm nền tảng cho phần thiết kế và triển khai hệ thống ở các chương sau.

CHƯƠNG 3. THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

3.1 Thiết kế kiến trúc

3.1.1 Lựa chọn kiến trúc phần mềm

Hệ thống được xây dựng theo kiến trúc phân tầng kết hợp mô hình “Simulation–Visualization” (mô phỏng–hiển thị), trong đó phần mô phỏng giao thông chạy trên SUMO (điều khiển bằng TraCI trong Python) và phần hiển thị/điều khiển chạy trên Unity (C#). Hai phần giao tiếp với nhau thông qua kết nối TCP socket theo cơ chế truyền thông điệp dạng JSON (có ký hiệu kết thúc gói tin <END> để tách thông điệp trên TCP stream).

Lý do lựa chọn kiến trúc này:

- **Tách biệt trách nhiệm:** SUMO xử lý mô phỏng vi mô (phương tiện, đèn giao thông, người đi bộ), Unity tập trung vào trực quan 3D, giao diện người dùng và tương tác.
- **Dễ mở rộng:** có thể bổ sung thêm loại đối tượng (xe buýt, xe ưu tiên), thêm chỉ số thống kê (thời gian hành trình, mật độ) mà không ảnh hưởng mạnh đến phần còn lại.
- **Tương thích công cụ:** SUMO đã có TraCI hỗ trợ điều khiển/đọc trạng thái theo bước mô phỏng; Unity thuận lợi cho hiển thị thời gian thực.

Trong cách ánh xạ theo hướng MVC ở mức khái niệm:

- **Model:** trạng thái mô phỏng trong SUMO (vehicle, traffic light, pedestrian) và các tệp mạng (.net.xml, .rou.xml, .sumocfg).
- **View:** các prefab/manager trong Unity để dựng hình đường, xe, đèn và hiển thị UI.
- **Controller:** lớp giao tiếp TCP và bộ xử lý lệnh (Pause/Resume/End) để đồng bộ mô phỏng.

3.1.2 Thiết kế tổng quan

Kiến trúc tổng quan gồm 3 tầng chính:

1. **Tầng mô phỏng (Python + SUMO/TraCI):** khởi tạo mạng giao thông, chạy mô phỏng theo step, trích xuất trạng thái và ghi thống kê kết quả.
2. **Tầng tích hợp (TCP + JSON protocol):** đóng gói dữ liệu trạng thái (đèn giao thông, xe, người đi bộ), xử lý phân mảnh/ghép gói trên TCP stream bằng marker <END>.

3. **Tầng hiển thị (Unity):** nhận dữ liệu theo thời gian thực, cập nhật/khởi tạo đối tượng trong scene; cung cấp UI cho người dùng điều khiển và kết thúc mô phỏng.

Hình minh họa kiến trúc tổng quan (tự vẽ theo mô tả):

Unity (C#) → [JSON/TCP]port 5050 Python Main → TraCI SUMO.

Unity gửi lệnh điều khiển (Pause/Resume/End) qua port điều khiển, Python phản hồi bằng dữ liệu trạng thái theo bước mô phỏng.

Hình 3.1: Kiến trúc tổng quan của hệ thống mô phỏng–hiển thị

3.1.3 Thiết kế chi tiết gói

Các gói/module chính và nhiệm vụ:

• **Python/HelloWorld:**

- `main.py`: điểm vào chương trình; khởi tạo server socket, nhận lệnh từ Unity, chạy vòng lặp TraCI và gửi dữ liệu trạng thái mỗi step.
- `network.py`: đóng gói giao thức truyền thông; đảm bảo tách thông điệp theo <END> để khắc phục đặc tính stream của TCP.
- `result.py`: ghi log hành trình xe ra CSV và tổng hợp thống kê ra JSON.
- `Traffic/*`: các module đọc trạng thái (xe, đèn, người đi bộ, đường) từ TraCI và chuẩn hóa về dạng JSON.
- `SUMO_xml/*`: sinh tệp cấu hình/mạng SUMO (node/edge/route/net/sumocfg), trong đó có chức năng sinh mạng từ mô cung.

• **Unity/Assets/Scripts:**

- `TrafficDataListener`: kết nối tới Python, gửi thông điệp `SimulationReady` và lắng nghe trạng thái mô phỏng theo thời gian thực.
- `RoadDataListener`: gửi `RoadDataRequest` để tải dữ liệu đường/nút giao một lần khi khởi động scene.
- `*Manager` (`VehicleManager`, `TrafficLightManager`, `PedestrianManager`): cập nhật/khởi tạo đối tượng dựa trên danh sách dữ liệu nhận được.
- `UI/OptionUI`: UI tùy chọn và gửi lệnh kết thúc mô phỏng (`SimulationEnd`).

3.2 Thiết kế chi tiết

3.2.1 Thiết kế giao diện

Ứng dụng hướng tới hiển thị mô phỏng thời gian thực trong Unity. Giao diện được thiết kế tối giản, tập trung vào:

- **Màn hình hiển thị chính:** camera quan sát mạng đường, xe và đèn giao thông; các đối tượng được cập nhật theo dữ liệu từ SUMO.
- **Bảng tùy chọn:** bật/tắt UI trong khi chạy, cho phép kết thúc mô phỏng an toàn (gửi lệnh về phía Python).

Giả định môi trường chạy phổ biến là PC, độ phân giải khuyến nghị 1920×1080 (Full HD) hoặc cao hơn; UI được thiết kế theo nguyên tắc dễ đọc, hạn chế chữ nhỏ và đảm bảo thao tác bằng chuột/bàn phím.

3.2.2 Thiết kế lớp

Trong phạm vi báo cáo, trình bày chi tiết một số lớp/module chủ đạo.

Nhóm lớp Python (mô phỏng và ghi log)

VehicleTripCsvLogger (trong `result.py`):

- Trách nhiệm: ghi nhận thời điểm xe xuất phát/đến đích theo step mô phỏng; ghi CSV và xuất thống kê trung bình ra JSON.
- Thuộc tính chính: `_depart_step_by_vehicle`, `_travel_steps_sum`, `_travel_steps_count`.
- Phương thức chính: `open()`, `on_step(step, departed, arrived)`, `close()`.

network.py (giao thức TCP):

- Trách nhiệm: tách/gộp thông điệp trên TCP bằng marker `<END>`; tránh lỗi khi một lần `recv` chứa nhiều thông điệp hoặc chỉ chứa một phần thông điệp.
- Hàm chính: `send_data()`, `receive_data()`, `receive_message()`, `server_thread()`.

Nhóm lớp Unity (nhận dữ liệu và cập nhật scene)

TrafficDataListener:

- Trách nhiệm: kết nối TCP đến Python, gửi `SimulationReady`, đọc JSON trạng thái; đẩy xử lý sang main thread để cập nhật scene an toàn.
- Phương thức chính: `StartListening()`, `ListenForData()`, `HandleData(json)`.

RoadDataListener:

- Trách nhiệm: gửi yêu cầu `RoadDataRequest` và nhận dữ liệu đường/nút giao 1 lần; lưu vào `RoadDataSO` để các hệ thống khác sử dụng.

TrafficLightManager:

- Trách nhiệm: quản lý tập đối tượng đèn giao thông trong scene; nếu đã tồn tại thì cập nhật trạng thái/vị trí, nếu chưa thì tạo mới từ prefab.

Luồng thông điệp (mô tả dạng trình tự)

Use case 1 — Unity yêu cầu dữ liệu đường (RoadData):

1. Unity khởi chạy `RoadDataListener` và mở kết nối TCP tới Python.
2. Unity gửi chuỗi `RoadDataRequest`.
3. Python trả về JSON mô tả edge/junction/crossing (đóng gói và kết thúc bằng `<END>`).
4. Unity parse JSON và lưu vào `RoadDataSO`; sau đó dừng listener.

Use case 2 — Đồng bộ trạng thái mô phỏng theo thời gian thực:

1. Unity gửi `SimulationReady` để bắt đầu vòng lặp mô phỏng.
2. Python chạy TraCI theo step; mỗi step đọc danh sách xe/đèn/người đi bộ và gửi về Unity.
3. Unity nhận JSON, phân rã thành danh sách và gọi các manager để cập nhật đối tượng trong scene.
4. Khi người dùng thoát ứng dụng, Unity gửi `Simulation end` để yêu cầu dừng mô phỏng.

3.2.3 Thiết kế cơ sở dữ liệu

Hệ thống không sử dụng cơ sở dữ liệu theo nghĩa truyền thống. Dữ liệu được lưu dưới dạng tệp:

- **Dữ liệu cấu hình/mạng mô phỏng:** các tệp XML của SUMO (`.nod.xml`, `.edg.xml`, `.net.xml`, `.rou.xml`, `.sumocfg`).
- **Dữ liệu kết quả thực nghiệm:** CSV ghi thời gian hành trình và JSON tổng hợp thống kê theo lần chạy (trong thư mục `result/`).

Thiết kế này phù hợp vì mục tiêu là mô phỏng/hiển thị và thu thập thống kê theo từng lần chạy, không cần truy vấn phức tạp hay đa người dùng.

3.3 Xây dựng ứng dụng

3.3.1 Thư viện và công cụ sử dụng

Bảng 3.1 liệt kê các công cụ/chính sách phiên bản sử dụng trong dự án.

Mục đích	Công cụ	Phiên bản/Ghi chú
Mô phỏng giao thông vi mô	Eclipse SUMO	1.24.0 (tạo file .sumocfg thể hiện version)
Điều khiển SUMO	TraCI (Python API)	đi kèm SUMO 1.24.0
Ngôn ngữ mô phỏng/tích hợp	Python	(điền theo máy: chạy python -version)
Trực quan 3D + UI	Unity Editor	6000.3.0f1
Parse JSON (Unity)	Newtonsoft.Json	com.unity.nuget.newtonsoft-json 3.2.2
Môi trường phát triển	Visual Studio Code	(điền theo máy: About VS Code)
Quản lý phiên bản	Git	(điền theo máy: git -version)
Hệ điều hành chạy thử nghiệm	Windows	(10/11 tùy máy triển khai)

Bảng 3.1: Danh sách thư viện và công cụ sử dụng

3.3.2 Kết quả đạt được

Các sản phẩm/dữ liệu đầu ra chính:

- **Bộ sinh bản đồ/mạng SUMO:** tạo node–edge từ bản đồ thành phố hoặc từ mê cung; sinh ra các tệp .nod.xml, .edg.xml, .net.xml, .rou.xml, .sumocfg.
- **Ứng dụng Unity hiển thị mô phỏng:** nhận dữ liệu thời gian thực từ Python và cập nhật đối tượng (xe, đèn, người đi bộ).
- **Bộ ghi log kết quả:** xuất CSV hành trình xe theo step và JSON tổng hợp theo từng lần chạy.

Thống kê nhanh mã nguồn (tính trên các phần do nhóm tự viết, loại trừ thư viện/.venv):

- Python: 20 file, 3045 dòng, dung lượng khoảng 109.5 KB.
- Unity C# (Assets/Scripts): 72 file, 4261 dòng, dung lượng khoảng 126.9 KB.
- Tệp cấu hình/mạng SUMO (HelloWorld/SUMO_xml): 6 tệp, tổng dung lượng khoảng 10.34 MB.

- Bản build Windows minh họa: `UnityBuild/TestGR1.1.exe` khoảng 0.64 MB (chưa tính toàn bộ thư mục build).

3.3.3 Minh họa các chức năng chính

Các chức năng chính đã thực hiện:

1. **Sinh mạng đường từ dữ liệu đầu vào:** hỗ trợ sinh mạng từ tệp bản đồ thành phố và sinh mạng từ mê cung (tạo node/edge theo cấu trúc lưới).
2. **Chạy mô phỏng SUMO bằng TraCI:** điều khiển mô phỏng theo step và đọc trạng thái xe/đèn/người đi bộ.
3. **Đồng bộ dữ liệu sang Unity theo thời gian thực:** gửi gói JSON theo TCP; Unity nhận và cập nhật scene.
4. **Điều khiển mô phỏng:** hỗ trợ tạm dừng/tiếp tục và kết thúc mô phỏng từ Unity.
5. **Xuất kết quả thực nghiệm:** ghi log hành trình xe và thống kê trung bình theo lần chạy.

Chèn ảnh minh họa 1: Scene Unity đang hiển thị xe/đèn giao thông theo dữ liệu SUMO.

Hình 3.2: Minh họa trực quan mô phỏng trên Unity

Chèn ảnh minh họa 2: SUMO-GUI đang chạy mô phỏng và cấu hình delay (ví dụ 110ms) để đồng bộ quan sát.

Hình 3.3: Minh họa mô phỏng trên SUMO-GUI

3.4 Kiểm thử

Do dự án có tính tích hợp cao (SUMO & Unity), kiểm thử được thực hiện theo hai hướng: (i) kiểm thử chức năng thử công theo use case và (ii) kiểm thử hộp đen cho các module xử lý dữ liệu/giao thức.

Các trường hợp kiểm thử tiêu biểu

Kết quả: các ca kiểm thử trên đạt yêu cầu trong điều kiện chạy thử nghiệm. Trường hợp sai lệch đồng bộ (giật/nhảy vị trí) chủ yếu xuất hiện khi delay quan sát trong SUMO-GUI đặt quá thấp; khi tăng delay (ví dụ khoảng 110ms) thì quan sát ổn định hơn.

TC	Mục tiêu	Dữ liệu vào / Kỳ vọng
TC01	Tách thông điệp TCP đúng theo <END>	Gửi liên tiếp 2 JSON trong 1 lần <code>sendall</code> ; kỳ vọng phía nhận tách đúng 2 thông điệp, không lẫn chuỗi.
TC02	Nhận RoadData một lần	Unity gửi <code>RoadDataRequest</code> ; kỳ vọng nhận JSON hợp lệ và lưu vào <code>RoadDataSO</code> , sau đó listener dừng.
TC03	Cập nhật trạng thái đèn giao thông	SUMO trả về chuỗi trạng thái <code>r/y/g/G</code> ; kỳ vọng Unity ánh xạ về trạng thái hiển thị tương ứng (đỏ/vàng/xanh).

Bảng 3.2: Một số trường hợp kiểm thử chính

3.5 Triển khai

Môi trường triển khai

Hệ thống được triển khai thử nghiệm trên máy cá nhân Windows với 2 tiến trình chính:

- **Python + SUMO:** chạy script chính để khởi tạo server và điều khiển TraCI.
- **Unity (Editor hoặc bản build):** chạy scene nhận dữ liệu và hiển thị.

Quy trình chạy thử nghiệm

Quy trình triển khai thử nghiệm (tóm tắt theo README):

1. Thêm dự án Unity vào Unity Hub và mở project.
2. Chạy script Python `main.py` để chuẩn bị kết nối và khởi chạy mô phỏng.
3. Khi SUMO-GUI mở, đặt delay quan sát phù hợp (ví dụ 110) và chạy mô phỏng.
4. Chạy Unity (Play mode hoặc bản build) để bắt đầu nhận dữ liệu và hiển thị.

Các cổng TCP sử dụng theo cấu hình hiện tại:

- Port 5050: Unity kết nối để nhận dữ liệu RoadData/TrafficData.
- Port điều khiển (ví dụ 5054): Unity gửi lệnh kết thúc mô phỏng (`Simulation end`).

Trong phạm vi thử nghiệm, hệ thống chạy ổn định theo mô hình “1 máy – 1 người dùng”; dữ liệu kết quả được lưu cục bộ thành CSV/JSON để phục vụ phân tích.

CHƯƠNG 4. CÁC GIẢI PHÁP VÀ ĐÓNG GÓP NỔI BẬT

Chương này trình bày các giải pháp kỹ thuật và đóng góp nổi bật của đề án trong quá trình xây dựng hệ thống hiển thị giao thông 3D từ dữ liệu mô phỏng SUMO. Các nội dung về kiến trúc và kết quả minh họa đã được trình bày ở chương trước (xem Hình 3.1); trong chương này, báo cáo tập trung vào những điểm mang tính “giải quyết vấn đề” và “đóng gói thành giải pháp” để có thể áp dụng lại cho các kịch bản khác.

4.1 Giải pháp giao tiếp thời gian thực qua TCP: tách khung thông điệp bằng marker <END>

4.1.1 Bài toán/vấn đề

Trong hệ thống, Unity cần nhận dữ liệu trạng thái theo từng bước mô phỏng (xe, đèn giao thông, người đi bộ) từ phía Python điều khiển SUMO. Một lựa chọn đơn giản là dùng TCP socket và gửi chuỗi JSON. Tuy nhiên, TCP là giao thức hướng luồng (stream-oriented), không bảo đảm ranh giới “mỗi lần gửi” tương ứng với “mỗi lần nhận”. Vì vậy, trong thực tế có thể xảy ra các tình huống:

- Một lần `recv` nhận được **một phần** thông điệp JSON (thiếu dữ liệu, JSON không parse được).
- Một lần `recv` nhận được **nhiều** thông điệp JSON gộp lại (Unity không biết tách ở đâu).

Ngoài ra, ứng dụng cần hỗ trợ các lệnh điều khiển từ Unity (Pause/Resume/End) và có thể cập nhật một số tham số chạy mô phỏng (ví dụ `timeStep`). Điều này đòi hỏi cơ chế nhận thông điệp điều khiển ổn định, tránh “kẹt” luồng nhận và đảm bảo đóng kết nối gọn.

4.1.2 Giải pháp

Đề án sử dụng TCP socket để truyền thông điệp dạng JSON giữa Python và Unity, đồng thời bổ sung cơ chế **tách khung** (message framing) bằng marker <END>.

Nguyên tắc đóng gói. Mỗi payload JSON được gửi như một chuỗi UTF-8, sau đó gửi thêm marker <END> để biểu thị kết thúc bản tin. Khi payload lớn, hàm `send_data` trong `network.py` chia chuỗi JSON thành các đoạn (chunk) theo kích thước bộ đệm và `sendall` tuần tự; marker luôn được gửi ở cuối để phía nhận biết điểm kết thúc.

Nguyên tắc bóc tách ở phía nhận. Phía nhận không parse theo từng `recv` mà

duy trì một *receive buffer* cho mỗi socket. Hàm `receive_message/receive_data` đọc thêm dữ liệu cho tới khi gặp marker `<END>`, sau đó tách ra đúng **một** thông điệp hoàn chỉnh và giữ lại phần dư cho lần nhận tiếp theo. Thiết kế buffer theo socket giải quyết đúng bản chất “TCP là stream”, tránh cả hai lỗi: JSON bị cắt giữa chừng và nhiều JSON bị dính lại.

Tích hợp điều khiển mô phỏng. Trên nền giao thức này, chương trình hỗ trợ hai nhóm bản tin:

- **Bản tin dữ liệu:** trạng thái theo step gồm `vehicles`, `trafficLights`, `pedestrians`.
- **Bản tin điều khiển:** các lệnh `Pause`, `Resume`, `Simulation end` và (tùy chọn) một JSON cấu hình đơn giản để cập nhật `timeStep`.

4.1.3 Kết quả đạt được

Giải pháp tách khung bản tin giúp kênh giao tiếp Unity–Python ổn định và dễ mở rộng hơn trong quá trình mô phỏng thời gian thực. Cụ thể:

- Giảm lỗi parse JSON do phân mảnh/gộp gói trên TCP stream.
- Cho phép truyền đồng thời dữ liệu trạng thái và lệnh điều khiển theo một cơ chế nhận thống nhất.
- Hỗ trợ tạm dừng/tiếp tục và kết thúc mô phỏng an toàn từ Unity, tạo trải nghiệm vận hành ổn định khi thử nghiệm.

4.2 Giải pháp điều phối vòng lặp mô phỏng, đa cổng kết nối và dừng hệ thống an toàn

4.2.1 Bài toán/vấn đề

Hệ thống mô phỏng–hiển thị gồm nhiều tác vụ chạy song song: (i) chạy mô phỏng SUMO theo step, (ii) gửi dữ liệu trạng thái cho Unity theo thời gian thực, (iii) nhận lệnh điều khiển (`Pause/Resume/End`), và (iv) (tùy chọn) nhận cập nhật đối tượng từ Unity. Trong bối cảnh này, hai nhóm vấn đề chính cần giải quyết là:

- **Tránh treo (deadlock) do I/O chặn:** các lời gọi `accept/recv` có thể chặn vô thời hạn, khiến chương trình không phản ứng khi cần dừng.
- **Dừng hệ thống gọn:** khi kết thúc mô phỏng cần đảm bảo đóng socket, dừng thread và ghi kết quả (CSV/JSON) đầy đủ; tránh trạng thái dở dang hoặc rò rỉ tài nguyên.

4.2.2 Giải pháp

Đồ án điều phối chương trình theo hướng đa luồng (multi-threading) và tách kênh kết nối theo mục đích, bám sát cách triển khai trong `main.py`:

- **Tách công theo kênh dữ liệu:** hệ thống khởi tạo ba server socket: port 5050 cho dữ liệu chính (RoadData/TrafficData), port 5053 cho kênh nhận cập nhật xe từ Unity (nếu dùng), và port 5054 cho kênh lệnh điều khiển (shutdown/pause/resume).
- **Cơ chế dừng tập trung:** sử dụng cờ `stop_event` để báo dừng cho toàn hệ thống; khi Unity gửi `Simulation end` hoặc khi mô phỏng kết thúc tự nhiên, cờ này được set và vòng lặp chờ ở tiến trình chính thoát.
- **Tạm dừng/tiếp tục an toàn:** dùng `pause_event` kết hợp với `sleep` để tạm dừng vòng lặp `simulationStep` mà không phá trạng thái TraCI.
- **Cập nhật tham số thời gian chạy:** `time_step` được bảo vệ bởi `time_step_lock` để cập nhật từ lệnh điều khiển, tránh race condition khi thread mô phỏng đang đọc giá trị này.
- **Shutdown gọn:** khi dừng, chương trình đóng các socket server, sau đó `join` các thread nền với timeout; luồng server trong `network.py` cũng có xử lý trường hợp socket đã bị đóng (trên Windows thường gặp `winerror 10038`) để thoát vòng `accept`.

4.2.3 Kết quả đạt được

Giải pháp điều phối giúp chương trình vận hành ổn định hơn trong quá trình thử nghiệm, đặc biệt ở các thao tác thường gặp như dừng mô phỏng hoặc tạm dừng để quan sát. Việc dừng gọn cũng đảm bảo các bước hậu xử lý (đóng TraCI, ghi file CSV/JSON tổng hợp) được thực hiện đầy đủ, qua đó tăng tính lặp lại của thực nghiệm.

4.3 Giải pháp sinh mạng SUMO để benchmark theo phương pháp vét cạn từ lưới .

4.3.1 Bài toán/vấn đề

Trong bối cảnh kiểm thử và đo đặc hiệu năng (benchmark), bài toán quan trọng không chỉ là “có bản đồ để chạy” mà còn là tạo được một mạng có cấu trúc **để điều khiển và có thể mở rộng kích thước** nhằm quan sát tải hệ thống (số lượng node/edge tăng theo kích thước bản đồ). Nếu phụ thuộc vào bản đồ thật (OpenStreetMap) hoặc dựng thủ công, việc đảm bảo tính lặp lại và điều chỉnh độ phức tạp của mạng sẽ khó và tốn thời gian.

Đồ án sử dụng đầu vào dạng lưới ký tự (@ là tường, . là lối đi). Tuy nhiên, thay

vì chỉ sinh node tại các “nút giao” suy ra từ hình học mê cung, dự án hiện tại chọn cách **vét cạn**: mỗi ô . được coi là một node để tạo ra mạng dày (dense) phục vụ benchmark.

Từ biểu diễn này, cần tạo được các tệp đầu vào chuẩn của SUMO:

- Vị trí các nút (junction) hợp lệ để tạo `.nod.xml`.
- Các cạnh có hướng (edge) kết nối các nút để tạo `.edg.xml`.
- Tệp kết nối/crossing (`.con.xml`) để hỗ trợ người đi bộ và các điểm qua đường.
- Quy tắc làn (lane) cho xe và người đi bộ để phục vụ hiển thị đa đối tượng.

4.3.2 Giải pháp

Trong dự án hiện tại, giải pháp sinh bản đồ benchmark được hiện thực trong `naive_map_creator.py` và dùng các hàm ghi XML trong thư mục `SUMO_xml`. Quy trình chính:

(1) Đọc lưới và vét cạn node. Hàm `parse_map_file` đọc phần header (width, height) và ma trận grid. Sau đó `get_node_pos_map` duyệt toàn bộ lưới; với mỗi ô ., tạo một node `n_i` và lưu vào `pos_map` theo khoá tọa độ làm tròn 2 chữ số (ổn định khi so khớp).

(2) Sinh cạnh theo 4 hướng. Để tạo `.edg.xml`, hàm `write_maze_edges_to_xml` (trong `SUMO_xml/create_map_from_maze.py`) duyệt từng node và quét theo 4 hướng (phải, xuống, trái, lên) cho tới khi gặp tường @. Node đầu tiên gặp được theo hướng đó sẽ được nối bằng một cạnh có hướng. Khi mọi ô . đều là node, phép quét này tương đương với việc nối các ô lối đi kề nhau, tạo ra một mạng dày để benchmark.

(3) Ghi file node/crossing và chuẩn hóa làn. `write_nodes_to_xml` ghi `.nod.xml` và sử dụng hệ số tỉ lệ (ví dụ `scale=40.0`) để chuyển tọa độ lưới sang đơn vị mét trong SUMO. Đồng thời `write_crossings_to_con_xml` sinh `.con.xml` để hỗ trợ người đi bộ. Khi ghi edge, cấu trúc làn được chuẩn hóa theo hướng có làn đi bộ (`allow="pedestrian"`) và các làn xe (`disallow="pedestrian"`) nhằm phục vụ hiển thị đa đối tượng.

Ghi chú về các phương án không vét cạn. Trong thư mục `SUMO_xml` còn có các thuật toán sinh bản đồ khác như `create_map_from_maze.py` (suy luận node từ hình học tường/độ bậc để tạo mạng thưa hơn) và `create_city_map.py` (sinh bản đồ theo kiểu “thành phố” với các heuristic kiểm tra kết nối). Các phương án này có mã nguồn và có thể dùng cho kịch bản thực tế hơn, nhưng **không được**

sử dụng trong pipeline benchmark hiện tại (xem cách gọi trong `main.py`).

4.3.3 Kết quả đạt được

Giải pháp vét cạn giúp sinh nhanh các mạng benchmark có độ phức tạp điều chỉnh được theo kích thước lưới, nhờ đó:

- Tạo mạng dày và có tính lặp lại cao để đo tải hệ thống (số lượng node/edge tăng theo kích thước bản đồ).
- Duy trì cú pháp dữ liệu SUMO chuẩn (node/edge/connection) để tiếp tục dùng `netconvert` và các công cụ chuẩn.
- Bảo đảm có làn người đi bộ và cấu trúc dữ liệu phù hợp để hệ thống hiển thị 3D biểu diễn đa đối tượng.

4.4 Giải pháp sinh tuyến đường (route) và lọc cặp điểm không liên thông

4.4.1 Bài toán/vấn đề

Sau khi có mạng đường, để tạo mô phỏng cần sinh tệp tuyến đường `.rou.xml`. Nếu chọn cặp điểm đầu–cuối (OD pair) ngẫu nhiên mà không kiểm tra tính liên thông, SUMO có thể không tìm thấy đường đi, dẫn tới lỗi hoặc làm giảm chất lượng kịch bản thử nghiệm. Bên cạnh đó, cần tạo kịch bản đa dạng (đi ngang mạng, đi từ trong ra ngoài, v.v.) để quan sát các tình huống giao thông khác nhau.

4.4.2 Giải pháp

Đề án xây dựng bộ sinh tuyến trong `SUMO_xml/route_gen.py` theo hai ý chính:

- **Chiến lược chọn OD pair:** các hàm `cross_side_gen`, `swap_side_gen`, `in_out_gen`, `out_in_gen` chia tập junction theo thống kê tọa độ (theo trục x hoặc theo bán kính so với tâm) để tạo các cặp OD có đặc trưng mong muốn.
- **Lọc tính khả thi:** trước khi ghi flow, hàm `filter_feasible_pairs` xây dựng đồ thị kề từ file edge và dùng BFS để kiểm tra từ điểm xuất phát có thể tới điểm đích hay không. Những cặp không khả thi được loại bỏ.

Với người đi bộ, hệ thống có thể sinh flow riêng và gán tham số `impatience` để thay đổi hành vi chờ đợi tại crossing.

Trong quá trình chạy mô phỏng, đề án sử dụng tùy chọn `-junction-taz` khi khởi chạy SUMO để cho phép coi ID của junction là điểm đầu/điểm cuối hợp lệ của chuyến đi. Thiết lập này phù hợp với cách sinh tuyến dựa trên cặp nút giao trong file route.

4.4.3 Kết quả đạt được

Giải pháp giúp giảm đáng kể tình huống “không tìm được đường đi” khi sinh tuyến tự động, đồng thời tạo được nhiều kiểu kịch bản (đi chéo mạng, đi xuyên tâm, v.v.) phục vụ kiểm thử hiển thị. Việc lọc khả thi cũng làm cho quá trình thử nghiệm có tính lặp lại cao hơn vì tránh phụ thuộc vào may rủi của chọn ngẫu nhiên.

4.5 Giải pháp chuẩn hóa dữ liệu TraCI cho hiển thị 3D (xe, đèn giao thông, người đi bộ)

4.5.1 Bài toán/vấn đề

TraCI cung cấp trạng thái mô phỏng theo thời gian (vị trí, tốc độ, góc hướng, trạng thái đèn, ...). Tuy nhiên, dữ liệu thô từ SUMO cần được chuẩn hóa để Unity có thể sử dụng nhất quán trong cập nhật 3D. Các vấn đề chính gồm:

- Dữ liệu của các loại đối tượng khác nhau cần được đưa về một “hợp đồng dữ liệu” thống nhất (ID, type, position, direction/forward, state).
- Với xe, cần suy ra vector hướng từ góc phương vị để xoay mô hình 3D đúng chiều.
- Với đèn giao thông, cần xác định vị trí đại diện của tín hiệu dựa trên hình học làn đường (lane shape).

4.5.2 Giải pháp

Đồ án xây dựng các hàm đọc dữ liệu trong thư mục HelloWorld/Traffic và đóng gói lại thành JSON theo cấu trúc rõ ràng:

- **Xe:** `read_vehicles` đọc vị trí (x, y) , tốc độ, lane ID và trạng thái xi-nhan/phanh. Góc hướng θ (độ) được chuyển sang vector đơn vị $\vec{f} = (\cos \theta, \sin \theta)$ (sau khi đổi về radian) để Unity có thể xoay xe.
- **Đèn giao thông:** `read_traffic_lights` đọc chuỗi trạng thái RYG của từng traffic light và danh sách lane được điều khiển. Với mỗi lane, lấy shape và chọn điểm cuối làm vị trí đặt đèn (ở mức trừu tượng). Trạng thái tín hiệu được chuyển thành 3 biến boolean tương ứng `isRed`, `isYellow`, `isGreen` (trong đó `g` và `G` đều được coi là xanh) để Unity hiển thị.
- **Người đi bộ:** `read_pedestrians` (tương tự) đọc danh sách ID và vị trí, phục vụ hiển thị.

Tại vòng lặp mô phỏng, dữ liệu được nhóm thành một payload gồm `trafficLights`, `vehicles`, `pedestrians` và gửi sang Unity theo cơ chế ở Mục 4.1.

Về ánh xạ hệ tọa độ, SUMO biểu diễn vị trí trên mặt phẳng (x, y) . Khi đưa sang Unity 3D, dữ liệu thường được chuyển thành (x, y_{unity}, z) , trong đó z lấy từ

y của SUMO và y_{unity} là độ cao (ví dụ một hằng số để đặt đối tượng lên mặt đường). Cách đặt này cũng phù hợp với cách xuất dữ liệu đèn giao thông trong `read_traffic_lights` khi sử dụng `position=[x, 5, y]`.

4.5.3 Kết quả đạt được

Giải pháp giúp Unity nhận dữ liệu có cấu trúc thống nhất và đủ thông tin để:

- Tạo/cập nhật đối tượng theo ID (vòng đời: xuất hiện–cập nhật–biến mất).
- Cập nhật chuyển động phương tiện theo vị trí và hướng.
- Hiển thị trạng thái đèn giao thông theo thời gian.

Đây là bước trung gian quan trọng để biến dữ liệu mô phỏng (mang tính kỹ thuật) thành dữ liệu hiển thị (mang tính trực quan) mà không phụ thuộc chặt vào cấu trúc nội bộ của SUMO.

4.6 Giải pháp ghi log và tổng hợp kết quả tự động theo lần chạy

4.6.1 Bài toán/vấn đề

Để đánh giá kịch bản mô phỏng và so sánh các cấu hình, hệ thống cần cơ chế ghi nhận kết quả theo từng lần chạy. Nếu chỉ quan sát trực quan, việc kết luận dễ mang tính cảm nhận và khó lặp lại. Mặt khác, dữ liệu đầu ra của SUMO có thể rất lớn; đồ án cần một cách ghi log gọn, tập trung vào chỉ số cơ bản phù hợp với phạm vi thử nghiệm.

4.6.2 Giải pháp

Đồ án xây dựng lớp `VehicleTripCsvLogger` trong `result.py` để ghi thông tin hành trình của xe theo đơn vị step mô phỏng. Ý tưởng:

- Ở mỗi step, lấy danh sách xe khởi hành và xe đến đích từ TraCI.
- Lưu lại `depart_step` theo `vehicle_id`; khi xe đến đích, tính `travel_steps` và ghi ra CSV.
- Kết thúc mô phỏng, tự động ghi một dòng tổng hợp (trung bình số step) và xuất thêm một file JSON tóm tắt (số xe, số người đi bộ quan sát được, `avg_vehicle_travel_steps`, thông tin map và tham số `pedestrian_impatience` nếu có).

Tên file kết quả được gắn timestamp để tránh ghi đè và thuận tiện truy vết theo lần chạy.

Để hạn chế mất dữ liệu khi dừng đột ngột, logger thực hiện `flush` sau khi ghi mỗi step và ghi thêm một dòng tổng hợp ở cuối file (`avg`). Đồng thời, file JSON tổng hợp được ghi riêng để Unity hoặc các công cụ phân tích có thể đọc nhanh mà

không cần duyệt toàn bộ CSV.

4.6.3 Kết quả đạt được

Giải pháp tạo ra bộ dữ liệu đầu ra nhẹ, dễ dùng cho phân tích sau mô phỏng:

- CSV phục vụ kiểm tra theo từng xe (depart/arrive/travel).
- JSON phục vụ tổng hợp nhanh hoặc dùng làm đầu vào cho các công cụ thống kê.

Nhờ đó, hệ thống không chỉ “hiển thị được” mà còn hỗ trợ đánh giá định lượng ở mức cơ bản, tạo nền tảng cho các hướng phát triển về sau (ví dụ phân tích theo tuyến, theo thời gian, hoặc theo khu vực).

4.7 Kết chương

Chương này đã trình bày các giải pháp và đóng góp nổi bật mang tính kỹ thuật của đồ án, tập trung vào các điểm then chốt để hiện thực hệ thống mô phỏng–hiển thị: (i) giao tiếp TCP đáng tin cậy bằng cơ chế tách khung bản tin, (ii) sinh mạng SUMO từ mô cung và chuẩn hóa làn cho đa đối tượng, (iii) sinh tuyến và lọc cặp OD không khả thi, (iv) chuẩn hóa dữ liệu TraCI cho Unity, và (v) ghi log/tổng hợp kết quả theo lần chạy. Các giải pháp này giúp hệ thống vận hành ổn định hơn và giảm đáng kể công sức chuẩn bị dữ liệu, đồng thời là cơ sở để mở rộng thêm chức năng trong phần hướng phát triển.

CHƯƠNG 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1 Kết luận

Trong phạm vi đề tài, đồ án đã xây dựng được một quy trình mô phỏng–hiển thị trong đó SUMO đảm nhiệm mô phỏng vi mô, Python điều khiển mô phỏng theo từng bước thông qua TraCI và Unity đảm nhiệm hiển thị không gian 3D theo thời gian thực. So với các cách quan sát kết quả mô phỏng truyền thống (thường tập trung vào 2D hoặc phụ thuộc nhiều vào thao tác dựng cảnh/đường đi thủ công), hệ thống của đồ án tập trung vào khả năng tự động hóa luồng dữ liệu và đồng bộ theo bước mô phỏng, qua đó hỗ trợ quan sát trực quan hơn trong các kịch bản có mật độ đối tượng cao hoặc khi cần thuyết minh kết quả.

Về kết quả đạt được, đồ án hoàn thành các nhóm chức năng chính sau:

- **Kết nối và đồng bộ thời gian thực:** triển khai kênh giao tiếp Unity–Python qua TCP/JSON và cơ chế tách khung bản tin bằng marker <END> để khắc phục đặc thù TCP hướng luồng; hỗ trợ bản tin dữ liệu và bản tin điều khiển (tạm dừng/tiếp tục/kết thúc).
- **Điều phối mô phỏng ổn định:** tổ chức đa luồng cho vòng lặp mô phỏng, các kênh nhận lệnh và (tùy chọn) kênh cập nhật từ Unity; bổ sung cơ chế dừng gọn để đảm bảo đóng kết nối và ghi kết quả đầy đủ.
- **Sinh dữ liệu bản đồ phục vụ benchmark:** xây dựng cách tạo mạng từ lưới ký tự theo hướng vét cạn (mỗi ô . là một node) để dễ mở rộng kích thước và đánh giá tải hệ thống; các thuật toán sinh mạng khác (ví dụ từ mê cung hoặc từ bản đồ thành phố) được giữ dưới dạng mã nguồn tham khảo trong thư mục SUMO_xml.
- **Chuẩn hoá dữ liệu trạng thái để hiển thị 3D:** trích xuất trạng thái xe/đèn giao thông/người đi bộ theo từng bước mô phỏng và chuyển đổi sang cấu trúc dữ liệu phù hợp để Unity dựng cảnh và cập nhật đối tượng.
- **Ghi log và tổng hợp thực nghiệm:** lưu thông tin di chuyển của xe theo định dạng bảng (CSV) và tạo thống kê tổng hợp theo lần chạy (JSON), phục vụ đối chiếu và đánh giá hiệu năng/độ ổn định.

Bên cạnh các kết quả đạt được, hệ thống vẫn còn một số hạn chế cần tiếp tục hoàn thiện. Thứ nhất, giao thức truyền thông hiện tại ưu tiên tính đơn giản (JSON) nên khi số lượng đối tượng lớn có thể phát sinh tải CPU/băng thông và làm giảm tốc độ cập nhật. Thứ hai, phần hiển thị trong Unity chủ yếu tập trung vào dựng và cập nhật trạng thái cơ bản; các yếu tố tăng tính chân thực (nội suy chuyển động,

mô hình hành vi, hiệu ứng/điều khiển chi tiết theo làn) chưa được khai thác đầy đủ. Thứ ba, dù trong mã nguồn có nhiều hướng sinh bản đồ (mê cung, bản đồ thành phố), cấu hình chạy thực nghiệm hiện tại ưu tiên mạng vết cặn để benchmark, do đó tính “thực tế” về hình học mạng chưa phải trọng tâm.

Từ quá trình thực hiện, đồ án rút ra một số kinh nghiệm chính: (i) trong hệ thống thời gian thực, cần thiết kế cơ chế phân định thông điệp rõ ràng khi dùng TCP để tránh lỗi khó truy vết; (ii) việc chuẩn hoá dữ liệu và thống nhất hệ toạ độ/đơn vị ngay từ đầu giúp giảm đáng kể công sức tích hợp giữa mô phỏng và hiển thị; và (iii) cơ chế dừng gọn và ghi log có ý nghĩa quan trọng để tăng tính lặp lại của thực nghiệm.

5.2 Hướng phát triển

Trong tương lai, đồ án có thể được phát triển theo các hướng chính sau:

- **Tối ưu truyền thông và hiệu năng:** giảm chi phí serialize/parse bằng cách gom gói theo cấu trúc nhị phân, nén dữ liệu, hoặc chỉ gửi phần thay đổi theo bước; bổ sung cơ chế đo độ trễ đầu-cuối để đánh giá chất lượng đồng bộ.
- **Mở rộng nguồn sinh bản đồ:** đưa các thuật toán sinh mạng không vết cặn (ví dụ rút gọn node theo nút giao, hoặc sinh mạng theo mẫu thành phố) thành tùy chọn cấu hình chạy; bổ sung pipeline nhập mạng từ OpenStreetMap/netconvert để tăng khả năng ứng dụng thực tế.
- **Nâng cấp hiển thị 3D:** cải thiện nội suy chuyển động và điều khiển quay/đổi làn để tăng độ mượt; bổ sung cơ chế camera theo dõi, hiển thị chú giải và các lớp thông tin (mật độ, tốc độ trung bình) nhằm phục vụ phân tích.
- **Bổ sung kịch bản và đánh giá:** xây dựng bộ kịch bản benchmark chuẩn theo nhiều kích thước mạng và mật độ phương tiện; chuẩn hoá thước đo (FPS, độ trễ, tỉ lệ mất gói, thời gian di chuyển) và tự động hoá chạy thực nghiệm.
- **Cải thiện chất lượng phần mềm:** bổ sung kiểm thử cho các mô-đun sinh mạng, sinh route và đóng gói giao thức; chuẩn hoá cấu hình chạy và tài liệu hoá giao thức để thuận tiện bảo trì/mở rộng.

MỘT SỐ LƯU Ý VỀ TÀI LIỆU THAM KHẢO

Lưu ý: Sinh viên không được đưa bài giảng/slide, các trang Wikipedia, hoặc các trang web thông thường làm tài liệu tham khảo.

Một trang web được phép dùng làm tài liệu tham khảo **chỉ khi** nó là công bố chính thống của cá nhân hoặc tổ chức nào đó. Ví dụ, trang web đặc tả ngôn ngữ XML của tổ chức W3C <https://www.w3.org/TR/2008/REC-xml-20081126/> là TLTK hợp lệ.

Có năm loại tài liệu tham khảo mà sinh viên phải tuân thủ đúng quy định về cách thức liệt kê thông tin như sau. Lưu ý: các phần văn bản trong cặp dấu < > dưới đây chỉ là hướng dẫn khai báo cho từng loại tài liệu tham khảo; sinh viên cần xóa các phần văn bản này trong ĐATN của mình.

<**Bài báo đăng trên tạp chí khoa học:** Tên tác giả, tên bài báo, tên tạp chí, volume, từ trang đến trang (nếu có), nhà xuất bản, năm xuất bản >

[1] E. H. Hovy, "Automated discourse generation using discourse structure relations," *Artificial intelligence*, vol. 63, no. 1-2, pp. 341–385, 1993

<**Sách:** Tên tác giả, tên sách, volume (nếu có), lần tái bản (nếu có), nhà xuất bản, năm xuất bản>

[2] L. L. Peterson and B. S. Davie, *Computer networks: a systems approach*. Elsevier, 2007.

[3] N. T. Hải, *Mạng máy tính và các hệ thống mở*. Nhà xuất bản giáo dục, 1999.

<**Tập san Báo cáo Hội nghị Khoa học:** Tên tác giả, tên báo cáo, tên hội nghị, ngày (nếu có), địa điểm hội nghị, năm xuất bản>

[4] M. Poesio and B. Di Eugenio, "Discourse structure and anaphoric accessibility," in *ESSLLI workshop on information structure, discourse structure and discourse semantics*, Copenhagen, Denmark, 2001, pp. 129–143.

<**Đồ án tốt nghiệp, Luận văn Thạc sĩ, Tiến sĩ:** Tên tác giả, tên đồ án/luận văn, loại đồ án/luận văn, tên trường, địa điểm, năm xuất bản>

[5] A. Knott, "A data-driven methodology for motivating a set of coherence relations," Ph.D. dissertation, The University of Edinburgh, UK, 1996.

<**Tài liệu tham khảo từ Internet:** Tên tác giả (nếu có), tựa đề, cơ quan (nếu có), địa chỉ trang web, thời gian lần cuối truy cập trang web>

[6] T. Berners-Lee, *Hypertext transfer protocol (HTTP)*. [Online]. Available:

`ftp://info.cern.ch/pub/www/doc/http-spec.txt.Z` (visited on 09/30/2010).

[7] Princeton University, *Wordnet*. [Online]. Available: `http://www.cogsci.princeton.edu/~wn/index.shtml` (visited on 09/30/2010).

TÀI LIỆU THAM KHẢO

- [1] E. H. Hovy, “Automated discourse generation using discourse structure relations,” *Artificial intelligence*, vol. 63, no. 1-2, pp. 341–385, 1993.
- [2] L. L. Peterson and B. S. Davie, *Computer networks: a systems approach*. Elsevier, 2007.
- [3] N. T. Hải, *Mạng máy tính và các hệ thống mở*. Nhà xuất bản giáo dục, 1999.
- [4] M. Poesio and B. Di Eugenio, “Discourse structure and anaphoric accessibility,” in *ESSLLI workshop on information structure, discourse structure and discourse semantics, Copenhagen, Denmark*, 2001, pp. 129–143.
- [5] A. Knott, “A data-driven methodology for motivating a set of coherence relations,” Ph.D. dissertation, The University of Edinburgh, UK, 1996.
- [6] T. Berners-Lee, *Hypertext transfer protocol (HTTP)*. Accessed: Sep. 30, 2010. [Online]. Available: <ftp://info.cern.ch/pub/www/doc/http-spec.txt.Z>
- [7] Princeton University, *Wordnet*. Accessed: Sep. 30, 2010. [Online]. Available: <http://www.cogsci.princeton.edu/~wn/index.shtml>

PHỤ LỤC

A. HƯỚNG DẪN VIẾT ĐỒ ÁN TỐT NGHIỆP

Quy định chung

Dưới đây là một số quy định và hướng dẫn viết đồ án tốt nghiệp mà bắt buộc sinh viên phải đọc kỹ và tuân thủ nghiêm ngặt.

Sinh viên cần đảm bảo tính thống nhất toàn báo cáo (font chữ, căn dòng hai bên, hình ảnh, bảng, margin trang, đánh số trang, v.v.). Để làm được như vậy, sinh viên chỉ cần sử dụng các định dạng theo đúng template ĐATN này. Khi paste nội dung văn bản từ tài liệu khác của mình, sinh viên cần chọn kiểu Copy là “Text Only” để định dạng văn bản của template không bị phá vỡ/vi phạm.

Tuyệt đối cấm sinh viên đạo văn. Sinh viên cần ghi rõ nguồn cho tất cả những gì không tự mình viết/vẽ lên, bao gồm các câu trích dẫn, các hình ảnh, bảng biểu, v.v. Khi bị phát hiện, sinh viên sẽ không được phép bảo vệ ĐATN.

Tất cả các hình vẽ, bảng biểu, công thức, và tài liệu tham khảo trong ĐATN nhất thiết phải được SV giải thích và tham chiếu tới ít nhất một lần. Không chấp nhận các trường hợp sinh viên đưa ra hình ảnh, bảng biểu tùy hứng và không có lời mô tả/giải thích nào.

Sinh viên tuyệt đối không trình bày ĐATN theo kiểu viết ý hoặc gạch đầu dòng. ĐATN không phải là một slide thuyết trình; khi người đọc không hiểu sẽ không có ai giải thích hộ. Sinh viên cần viết thành các đoạn văn và phân tích, diễn giải đầy đủ, rõ ràng. Câu văn cần đúng ngữ pháp, đầy đủ chủ ngữ, vị ngữ và các thành phần câu. Khi thực sự cần liệt kê, sinh viên nên liệt kê theo phong cách khoa học với các ký tự La Mã. Ví dụ, nhiều sinh viên luôn cảm thấy hối hận vì (i) chưa cố gắng hết mình, (ii) chưa sắp xếp thời gian học/chơi một cách hợp lý, (iii) chưa tìm được người yêu để chia sẻ quãng đời sinh viên vất vả, và (iv) viết ĐATN một cách cẩu thả.

Trong một số trường hợp nhất thiết phải dùng các bullet để liệt kê, sinh viên cần thống nhất Style cho toàn bộ các bullet các cấp mà mình sử dụng đến trong báo cáo. Nếu dùng bullet cấp 1 là hình tròn đen, toàn bộ báo cáo cần thống nhất cách dùng như vậy; ví dụ như sau:

- Đây là mục 1 – Thực sự không còn cách nào khác tôi mới dùng đến việc bullet trong báo cáo.
- Đây là mục 2 – Nghĩ lại thì tôi có thể không cần dùng bullet cũng được. Nên tôi sẽ xóa bullet và tổ chức lại hai mục này trong báo cáo của mình cho khoa học hơn. Tôi muốn thầy cô và người đọc cảm nhận được tâm huyết của tôi

trong từng trang báo cáo ĐATN.

A.1 Ngành học

Sinh viên lưu ý viết đúng ngành/chuyên ngành trên bìa và trên gáy theo đúng quy định của Trường. Ngành học hay chuyên ngành học phụ thuộc vào ngành học mà sinh viên đăng ký. Sinh viên có thể đăng nhập trên trang quản lý học tập của mình để xem lại chính xác ngành học của mình.

Một số ví dụ sinh viên có thể tham khảo dưới đây, trong trường hợp có chuyên ngành thì sinh viên không cần ghi chuyên ngành:

- Đối với kỹ sư chính quy:
 - Từ K61 trở về trước: Ngành Kỹ thuật phần mềm
 - Từ K62 trở về sau: Ngành Khoa học máy tính
- Đối với cử nhân:
 - Ngành Công nghệ thông tin
- Đối với chương trình EliteTech:
 - Chương trình Việt Nhật/KSTN: Ngành Công nghệ thông tin
 - Chương trình ICT Global: Ngành Information Technology
 - Chương trình DS&AI: Ngành Khoa học dữ liệu

A.2 Đánh dấu (bullet) và đánh số (numering)

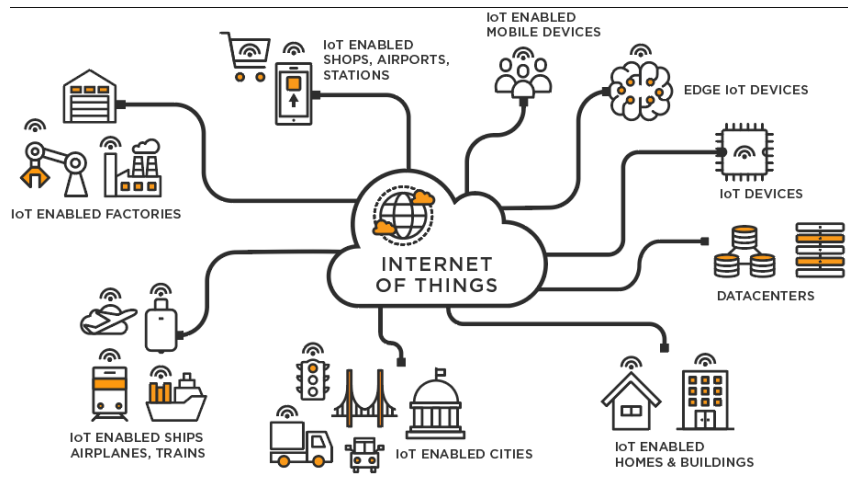
Việc sử dụng danh sách trong LaTeX khá đơn giản và không yêu cầu sinh viên phải thêm bất kỳ gói bổ sung nào. LaTeX cung cấp hai môi trường liệt kê đó là:

- Đánh dấu (bullet) là kiểu liệt kê không có thứ tự. Để sử dụng kiểu liệt kê đánh dấu, chúng ta khai báo như sau

```
\begin{itemize}
\item Nội dung thứ nhất được viết ở đây.
\item Nội dung thứ hai được viết ở đây.
\item ...
\end{itemize}
```

- Đánh số (numering) là kiểu liệt kê có thứ tự. Để sử dụng kiểu liệt kê đánh số, chúng ta khai báo như sau

```
\begin{enumerate}
\item Nội dung thứ nhất được viết ở đây.
\item Nội dung thứ hai được viết ở đây.
\item ...
\end{enumerate}
```



Hình A.1: Internet vạn vật

`\end{enumerate}`

Chú ý các nội dung trình bày trong cả hai môi trường liệt kê theo sau lệnh `\item`. Ngoài ra LaTeX còn cung cấp một số kiểu liệt kê khác, sinh viên có thể tham khảo tại <https://www.overleaf.com/learn/latex/Lists>

A.3 Cách thêm bảng

Col1	Col2	Col2	Col3
1	6	87837	787
2	7	78	5415
3	545	778	7507
4	545	18744	7560
5	88	788	6344

Bảng A.1: Table to test captions and labels.

Bảng A.1 là ví dụ về cách tạo bảng. Tất cả các bảng biểu phải được đề cập đến trong phần nội dung và phải được phân tích và bình luận. Chú ý: Tạo bảng trong Latex khá phức tạp và mất thời gian, vì vậy sinh viên có thể sử dụng các công cụ hỗ trợ tạo bảng (Ví dụ: <https://www.tablesgenerator.com/>). Sinh viên có thể tìm hiểu sâu hơn về cách chèn ảnh trong Latex tại link <https://www.overleaf.com/learn/latex/Tables>.

A.4 Chèn hình ảnh

Hình A.1 là ví dụ về cách chèn ảnh. Lưu ý chú thích của hình vẽ được đặt ngay dưới hình vẽ. Sinh viên có thể tìm hiểu sâu hơn về cách chèn ảnh trong Latex tại https://www.overleaf.com/learn/latex/Inserting_Images.

Chú ý, tất cả các hình vẽ phải được đề cập đến trong phần nội dung và phải được

phân tích và bình luận.

A.5 Tài liệu tham khảo

Cách liệt kê

Áp dụng cách liệt kê theo quy định của IEEE. Ví dụ của việc trích dẫn như sau **scott2013sdn**. Cụ thể, sinh viên sử dụng lệnh `\cite{}` như sau **ashton2009internet**. Chỉ những tài liệu được trích dẫn thì mới xuất hiện trong phần Tài liệu tham khảo. Tài liệu tham khảo cần có nguồn gốc rõ ràng và phải từ nguồn đáng tin cậy. Hạn chế trích dẫn tài liệu tham khảo từ các website, từ wikipedia.

Các loại tài liệu tham khảo

Các nguồn tài liệu tham khảo chính là sách, bài báo trong các tạp chí, bài báo trong các hội nghị khoa học và các tài liệu tham khảo khác trên internet.

A.6 Cách viết phương trình và công thức toán học

Các gói `amsmath`, `amssymb`, `amsfonts` hỗ trợ viết phương trình/công thức toán học đã được bổ sung sẵn ở phần đầu của file `main.tex`. Một ví dụ về tạo phương trình (A.1) như sau

$$F(x) = \int_b^a \frac{1}{3}x^3 \quad (\text{A.1})$$

Phương trình A.1 là ví dụ về phương trình tích phân. Một phương trình khác không được đánh số thứ tự (gán nhãn)

$$x[t_n] = \frac{1}{\sqrt{N}} \sum_{k=0}^{N-1} X[f_k] e^{j2\pi nk/N}$$

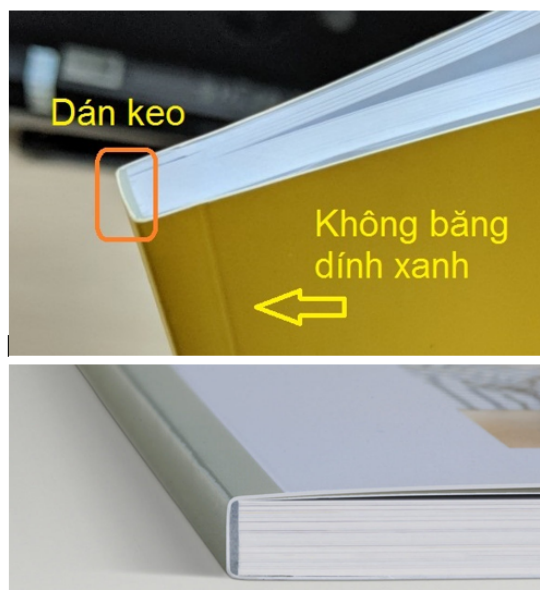
Phương trình này thể hiện phép biến đổi Fourier rời rạc ngược (IDFT).

A.7 Qui cách đóng quyển

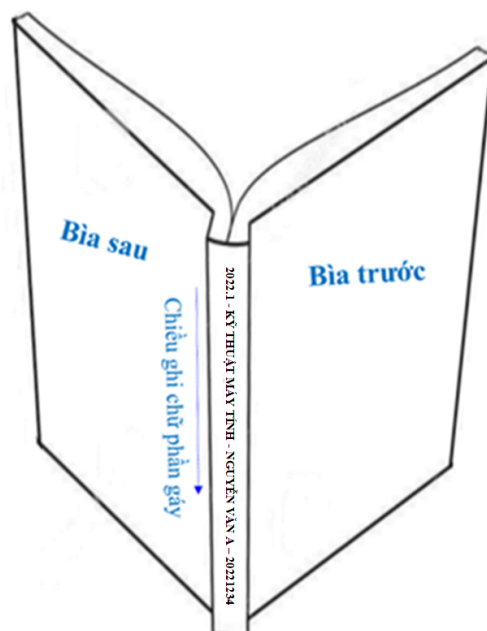
Phần bìa trước chế bản theo qui định; bìa trước và bìa sau là giấy liền khổ. Sử dụng keo nhiệt để dán gáy khi đóng quyển thay vì sử dụng băng dính và dập ghim như mô tả ở Hình A.3 Phần gáy ĐATN cần ghi các thông tin tóm tắt sau: Kỳ làm ĐATN - Ngành đào tạo - Họ và tên sinh viên - Mã số sinh viên. Ví dụ:

2022.1 - KỸ THUẬT MÁY TÍNH - NGUYỄN VĂN A - 20221234

Qui cách ghi chữ phần gáy như hình dưới đây:



Hình A.2: Qui cách đóng quyển đồ án



Hình A.3: Qui cách đóng quyển đồ án

B. ĐẶC TẢ USE CASE

Nếu trong nội dung chính không đủ không gian cho các use case khác (ngoài các use case nghiệp vụ chính) thì đặc tả thêm cho các use case đó ở đây.

B.1 Đặc tả use case “Thống kê tình hình mượn sách”

...

B.2 Đặc tả use case “Đăng ký làm thẻ mượn”

...